

SQL Test:-

Q1:-Ans → Select rider-type, Count(*) as total-rides,
Sum(fare) as total-fare from rides
Group by rider-type having total-rides
> 7 order by total-fare desc;

Q2 Ans → Select station-id, Count(*) as ride-count,
Round(Avg(duration-min), 2) as avg-duration
from rides group by station-id having
avg-duration > 30 order by avg-duration
desc;

Q.3 Ans → Select month(ride-date) as ride-month,
Count(*) as total-rides,
Round(Sum(distance-km), 1) as total-distance
from rides group by ride-month
having total-distance > 30 order by ride-
month;

Q4 Ans → Select rider-type, max(duration-min) as
longest-ride, min(duration-min) as shortest-
ride,
Round(Avg(duration-min), 2) as Avg-ride
from rides group by rider-type
having max(duration-min) >= 50
order by longest-ride desc;

Q.5 Ans → Select station-id, rider-type, Count(*)
as ride-count,
Round(Sum(distance-km), 1) as total-distance
from rides group by station-id, rider-type having
total-distance > 10 order by station-id;



22/10/22

Q6. Ans $\rightarrow$ Select station_id,
Count (distinct rider_type) as rider_type_count,
Count (*) as total_rides,
Round (avg (fare), 2) as Avg_fare from rides
group by station_id having Count (distinct
rider_type) = 3 order by station_id;

Q7. Ans: - select s.station_id, s.station_name,
s.zone, r.ride_id, r.rider_type, r.fare
from stations as s left join rides as r
on s.station_id = r.station_id
and month (r.ride_date) = 6
and year (r.ride_date) = 2024
order by s.station_id, r.ride_id;

Q8. Ans - Types of SQL languages are -

①. DDL (Data definition language)
Used to create or modify database
objects.

②. DML (Data Manipulation language)
Used to insert, update, delete records.

③. DQL (Data Query language):
Used to retrieve data.

④. DCL (Data Control language):
Used to manage transactions.



~~Q. 100ms~~ A join is used to combine rows from two or more tables based on a related column.

Types of Joins:

- ①. Inner Join - Return only matching rows from both tables.
- ②. Left join → Return all rows from left ~~side~~ table and matching rows from right table. If no match exists, they returns null values.
- ③. Right Join - Return all rows from the right table and matching rows from the left table.
- ④. Full outer Join - Returns all rows from both tables.
- ⑤. Cross Join - Returns the Cartesian product of both tables.

